Hackermail

A hackable email framework — design exploration

Living document · started 2026-05-21

Contents

Vision

The 10-year mantra

Design every abstraction today by asking: *what does this look like when the system is maxed out in 10 years?* Then strip back to the minimum viable shape that can morph into that endpoint without breaking changes. Authorization is the canonical example — we ship a stub today, but the seams must already be where a capability system will live.

One-line pitch

An email engine where *everything* — protocols, storage adapters, UI clients, AI agents, routing logic — is a plugin speaking a single JSON-schema'd wire protocol over HTTP/webhooks. The core owns only the email algebra and the extension contract.

Non-goals (today)

- A finished authorization system (stub only; seam ready).
- A reference UI (a UI is a plugin; we may ship one, but it is not core).
- Re-inventing JMAP — we steal from it heavily.

Core Principles

1. **Minimum fixed core.** Object model, event bus, plugin registry, capability seam. Nothing else.
2. **Plugin-first.** IMAP, SMTP, Gmail API, Outlook, storage backends, UIs, AI agents, routers — all plugins.
3. **One wire format.** JSON-schema-defined messages over HTTP + webhooks. No gRPC, no multi-transport matrix. In-process plugins use a thin shim that skips serialization but obeys the same schemas.
4. **Namespaced metadata, declared dependencies.** Every object carries a JSON blob partitioned by plugin namespace. Plugins *declare* which namespaces they read and write; the engine enforces and versions the schemas.
5. **Future-proof seams, not future features.** Build the seam, ship a stub.

Core Object Model

The fixed email algebra. Plugins extend via metadata namespaces; they do not add new core types without an RFC.

Naming convention

We align our type names with JMAP wherever the concept matches, so that the JMAP-server plugin is a near-trivial mapping and so that developers familiar with JMAP feel at home. `Email`, `EmailObject`, `EmailSubmission`, `Mailbox`, `Thread`, `Identity` all come from (or extend) JMAP. `Channel` and `Address` are our additions.

Entities

- **Account** — an identity within the system. May span multiple channels.
- **Channel** — a concrete protocol binding (e.g. IMAP-inbox-A, SMTP-out-B, Gmail-API-C). Channels are owned by plugins.
- **Address** — an RFC5322 address, mapped to one or more Accounts via routing.
- **Mailbox** — a logical container. Folder-like *or* label-like; the model must support both semantics.
- **Email** — the canonical, immutable email artifact: headers + body parts + metadata blob. Content-addressed.
- **EmailObject** — per-account mutable state for an `Email`: mailbox membership, keywords/flags, plugin annotations. (One `Email` may have N `EmailObject` rows, one per receiving Account.)
- **EmailSubmission** — an attempt to send. Has a lifecycle (see §Lifecycle).
- **Thread** — a derived grouping. Threading algorithm is itself a plugin.

- **Attachment** — referenced by `Email`, stored by a storage plugin.
- **Flag** — seen/unseen/starred/custom. Custom flags live in metadata.

Verbs (core operations)

`fetch`, `store`, `send`, `receive`, `move`, `copy`, `flag`, `search`, `subscribe`. Each verb is a wire-protocol method with a JSON Schema.

Notably absent: a single `delete`. "Delete" in email means too many different things (remove from one mailbox, move to trash, mark `\Deleted`, expunge, drop a label, destroy all copies). We split it into precise verbs to keep user intent legible end-to-end:

- `removeFromMailbox` — drop one `MailboxId` from `EmailObject.mailboxIds`.
- `archive` — opinionated alias: remove from Inbox (provider-specific mailbox identification is a routing plugin concern).
- `trash` — move to the account's Trash mailbox.
- `expunge` — permanently delete the `EmailObject` (and any `Email` rows it was the last reference to).
- `destroyEmail` — admin-only; destroys the immutable `Email` and all `EmailObject`s pointing to it. Requires elevated capability.

A `move` is canonical mailbox-set mutation on `EmailObject`. Whether the channel plugin implements it as IMAP `MOVE`, `COPY`+`STORE` `\Deleted`+`EXPUNGE`, or a Gmail label swap is the plugin's problem; the canonical operation is unambiguous.

Metadata blob

Every core entity carries:

```
{
  "core": { /* fixed schema */ },
  "ext": {
    "<plugin-namespace>": { /* plugin-defined, schema-versioned */ }
  }
}
```

Plugins declare in their manifest:

- `writes: ["my.namespace"]`
- `reads: ["other.plugin.namespace@>=1.2"]`

The engine enforces writes, version-checks reads, and refuses to load a plugin whose declared reads are unsatisfied.

Plugin System

Plugin types (non-exhaustive)

- **Protocol plugins** — IMAP, SMTP, JMAP, Gmail API, Microsoft Graph, MAPI, custom protocols.
- **Storage plugins** — email store, attachment store, index, cache.
- **Routing plugins** — address → account, catch-all, split send/receive, per-channel muxing.
- **Processing plugins** — threading, dedup, spam, encryption, signing.
- **Client plugins** — UIs (web, TUI, native), mobile sync endpoints.
- **Agent plugins** — AI auto-responders, classifiers, summarizers.

Plugin manifest (sketch)

```
{
  "name": "hackermail.imap",
```

```
  "version": "0.1.0",
  "capabilities": ["protocol.fetch", "protocol.receive", "mailbox.list"],
  "metadata": {
    "writes": ["hackermail.imap"],
    "reads":  ["hackermail.core@>=1"]
  },
  "transport": "http",
  "endpoint": "http://localhost:9101",
  "webhooks": ["email.received", "mailbox.changed"]
}
```

Wire protocol

- JSON over HTTP for request/response.
- Webhooks (HTTP POST from engine → plugin, and plugin → engine) for events.
- All payloads validated against JSON Schemas published by the engine and by plugins (for their namespaces).
- Schema registry is part of core; versioned; additive changes only within a major version.
- In-process plugins implement the same method signatures but skip HTTP/JSON encode-decode.

UI as plugin — why

A UI may want to annotate an email ("user dismissed this banner", "thread collapsed in this client"). It needs the same write-to-namespace ability as any other plugin. Treating UIs as plugins gives them this for free and keeps the engine from growing a UI-specific surface.

Client vs. extension — still separate concerns

Even though UIs are plugins, we keep two *conceptual* roles:

- **Extension role** — plugin extends engine behavior (handlers, hooks).
- **Consumer role** — plugin reads/writes data, subscribes to events.

A UI is mostly a consumer that happens to also extend (annotations). The wire protocol exposes both as the same surface; the distinction is documentation, not code.

Domain Model — Deep Dive

This section is the heart of the design. Everything else (plugins, wire protocol, routing, authz) is mechanism; the model is what we actually owe the world to get right.

What email really is

Email is not "messages in folders". It is a partially-replicated, eventually-consistent, append-mostly log of immutable artifacts (RFC5322 blobs), overlaid with mutable per-account state (flags, mailbox membership, labels, annotations). Different providers expose different *views* of this underlying reality, and the views disagree:

- **IMAP** — messages live *in* a mailbox (folder). Same message in two folders means two copies (different UIDs). State is server-side.
- **Gmail** — messages have *labels*; "folders" are a UI fiction. Same message in two labels is still one message. State is server-side.
- **JMAP** — messages have a set of mailbox ids (multi-membership), making labels and folders the same thing. State is server-side, queryable as one coherent model.
- **Local mbox / Maildir** — flat file, no server, state is filesystem.
- **Exchange/MAPI** — folders + categories + a much larger object graph (calendars, tasks) bolted on.

A hackable framework cannot pick one view and force the others into it. It must model the *underlying reality* and present each view as a *projection*.

The canonical model

We adopt a JMAP-shaped core because JMAP is the only standard that already models email as multi-membership + immutable-message + mutable- state. We extend it with explicit provenance and a metadata blob.

Entity	Role	JMAP equivalent
`Email`	Immutable RFC5322 artifact + parsed headers + parts.	`Email` (the immutable bits)
`EmailObject`	Per-account mutable state: flags, mailbox set, annotations.	`Email` (the mutable bits)
`Mailbox`	A named bucket. Folder-*or*-label by role/semantics flag.	`Mailbox`
`Thread`	Derived grouping of `Email`s by a pluggable algorithm.	`Thread`
`EmailSubmission`	An attempt to send. Has lifecycle: queued → sent → delivered/bounced.	`EmailSubmission`
`Identity`	A sending identity: address + display name + signature.	`Identity`
`Account`	A logical user-facing account. Owns Identities and Mailboxes.	`Account`
`Channel`	*New.* A concrete protocol binding (IMAP conn, Gmail OAuth, …).	— (we add this)
`Address`	*New.* RFC5322 address, routed to Account(s) by routing plugin.	— (we add this)

The two additions — `Channel` and `Address` — are where we diverge from JMAP. JMAP assumes a single server owns everything; we don't. `Channel` makes the provider binding explicit (so one Account can fan in from many sources), and `Address` makes routing first-class (so catch-all and split-send/receive are model-level, not config-level).

Email vs. EmailObject — why split

JMAP folds these together for wire convenience. Internally we split them because their lifecycles differ wildly:

- `Email` is *immutable* once received. It is content-addressed (hash of canonicalized RFC5322). Two accounts receiving the same message share one `Email` row.
- `EmailObject` is *per-account-per-email*, mutates constantly (read, flagged, moved), and is what UIs and agents actually scribble on.

This split also makes encryption-at-rest tractable: encrypt `Email` bodies with per-account keys, leave `EmailObject` (which has no body) queryable.

AnalyzedDocument — a derived, transient artifact

Distinct from both: the `AnalyzedDocument` is the normalized output of a single analysis pass over an `Email` — extracted plaintext per part, detected language, structured header fields, attachment-extracted text, and chunk boundaries. It is *derived* (a pure function of `Email` content
1. analyzer version, hence content-addressed and computed once, shared

across accounts) and *transient* (it is not canonical state; it exists to feed Index Providers — see §Storage). Because it is the *plaintext* of a possibly-encrypted body, it is `plaintext-tainted` and privileged exactly like a decrypted view (§Crypto): cacheable in the `cache` role only for non-encrypted

mail, and behind the privileged-search capability otherwise. It is the sibling of the decrypted view, never of canonical storage.

Mailbox: folder or label?

A `Mailbox` has a `semantics` field: `exclusive` (folder-like — an email in this mailbox is *not* in another exclusive mailbox of the same account) or `inclusive` (label-like — free multi-membership). Adapters declare which they emit.

This lets a Gmail adapter expose all-as-inclusive, an IMAP adapter expose all-as-exclusive, and a JMAP adapter mix freely. The UI can choose whether to render exclusive mailboxes as a tree and inclusive as chips.

Thread ownership

Threads are derived, but threads are referenced by ids in the UI and in metadata. Resolution: core owns thread *ids* (stable, opaque), a thread plugin owns thread *membership* (which `Email`s belong). Swapping the plugin re-derives membership but ids stay stable per (algorithm, account) pair. A migration tool can rewrite ids across an algorithm change.

Provenance

Every `Email` and every `EmailObject` mutation records a structured provenance record. The minimum schema is:

```
{
  "channel_id":  "imap_main",
  "plugin":      "hackermail.imap@0.3.1",
  "at":          "2026-05-22T08:14:03Z",
  "provider": {
    "mailbox":      "INBOX",
    "uidvalidity":  "1700001234",
    "uid":          "55821",
    "gm_msgid":     null,
    "gm_thrid":     null
  },
  "cause": "fetch" | "user.move" | "policy.archive" | "sync.repair" | ...
}
```

Provider-specific fields (UIDVALIDITY/UID for IMAP, X-GM-MSGID for Gmail, internetMessageId always) live under `provider`. The `cause` field records *why* this mutation happened, which is the difference between debuggable and undebuggable when state diverges.

This is the audit trail, the foundation for future authz enforcement, *and* the foundation for sync repair when UIDVALIDITY rolls or a provider re-keys.

It is non-optional: the engine refuses writes without provenance.

Identity & deduplication

Content-addressed `Email` rows must handle the fact that "the same message" rarely arrives byte-identical: `Received:` headers differ, DKIM signatures stack, mailing-list footers get added, MIME boundaries get rewritten, line endings get normalized, spam scanners inject headers. A single strict hash is wrong: too strict and duplicates don't collapse; too loose and distinct messages merge.

We carry *two* identifiers on every `Email`:

- `rawHash` — SHA-256 of the bytes as received. Strict. Used as primary key for the immutable artifact. Two arrivals that differ in even one `Received:` header are two `Email` rows.
- `fingerprint` — a soft, canonicalized hash: `Message-ID` if present and well-formed; otherwise a fallback over `(From, Date, normalized`

`Subject, body-content-hash with footers/signatures stripped`). Computed by a dedup plugin; algorithm-versioned.

The engine uses `fingerprint` to *link* `Email` rows it believes represent the same logical message, but does not collapse them. UIs and agents traverse the link to deduplicate at display time; the storage layer keeps the originals so that provenance, signatures, and provider round-trips remain intact.

This split also matters for Gmail-over-IMAP, where the same Gmail message appears in multiple IMAP folders as distinct copies. The dedup plugin uses `X-GM-MSGID` (when surfaced) to link them; without it, falls back to `fingerprint`.

Lifecycle & State Machines

Two machines, not one

A common mistake is to draw "received → read → replied → archived" as a single state machine on `Email`. It isn't. There are *two* distinct lifecycles, on *two* different entities, and conflating them produces a model where every operation has to special-case half the states.

1. **Outbound lifecycle.** Lives on `EmailSubmission`. Models the journey of an attempt to send: drafted, scheduled, queued, sending, sent, delivered/bounced, failed.
2. **Per-account treatment.** Lives on `EmailObject`. *Not* a state machine. A set of orthogonal flags (seen, flagged, answered, …) plus mailbox membership. Trying to force this into a state machine ("received → read → replied") collapses orthogonal axes and breaks the moment an email is both unread *and* replied (yes, this happens).

Why "received" is not a state

A received `Email` is an immutable artifact. The thing that varies is *which account has it* and *what flags that account has applied* — both captured by `EmailObject`. There is no "received" state because there is nothing else for a received email to become. Treating reception as event-not-state also matches reality: reception is a discrete event (webhook fires, `EmailObject` row is created), not a phase of existence.

Drafts: pre-EmailSubmission artifacts

A draft is an `Email` (often incomplete — no `Message-ID` yet, no `Date`) sitting in a Drafts mailbox with the `$draft` keyword, with *no* associated `EmailSubmission`. The moment the user clicks Send, an `EmailSubmission` is created referencing the draft `Email`. If they click Schedule, the same thing happens but with `sendAt` set in the future.

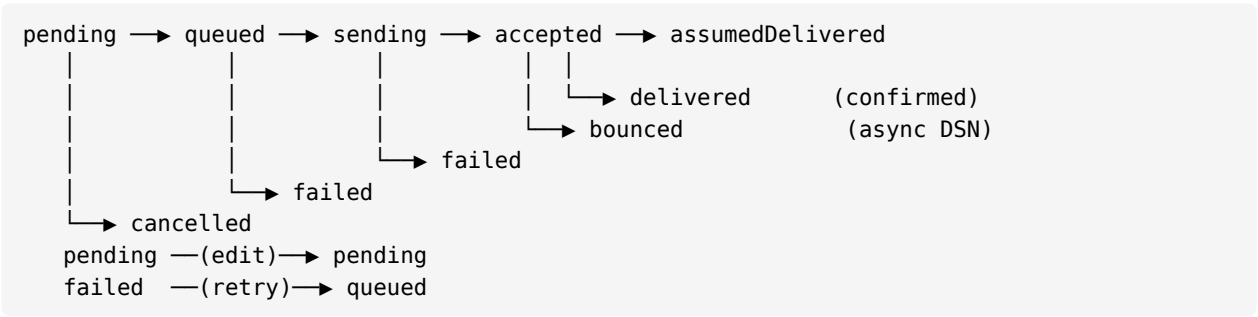
This means: *drafts are not a state of EmailSubmission, they are the absence of EmailSubmission.* Same for "scheduled but cancelled" — the `EmailSubmission` moves to `cancelled`, the underlying `Email` stays as a draft, ready to be re-submitted.

The EmailSubmission state machine

State	Meaning	Terminal?
`pending`	Created, waiting for `sendAt` (immediate or future).	no
`queued`	`sendAt` reached; handed to an outbound channel.	no
`sending`	Channel plugin is actively transmitting.	no
`accepted`	Next-hop server accepted (SMTP 2xx / provider 2xx). Not delivery.	no
`assumedDelivered`	No bounce after grace period (default 24h). Soft success.	yes
`delivered`	Confirmed delivery (DSN success, provider read-callback, etc.).	yes
`bounced`	Permanent delivery failure (DSN or async 5xx bounce email).	yes
`failed`	Transient retries exhausted, or local error.	yes

State	Meaning	Terminal?
`cancelled`	User or system cancelled before `sending`.	yes

Allowed transitions:



We deliberately do *not* overpromise `delivered`. SMTP `accepted` means the next hop took responsibility, nothing more. UIs should display `accepted` as the realistic terminal for most sends, `delivered` as a bonus signal when DSNs / provider APIs / read receipts give us one.

Invariants:

- Transitions are append-only; we never overwrite an `EmailSubmission` row, we add an `EmailSubmissionEvent`. The current state is the latest event. This gives us a free audit log and makes provenance trivial.
- Only `pending` is mutable from the client (edit subject, body, sendAt, cancel). Everything past `queued` is system-driven.
- An `EmailSubmission` references exactly one `Email`. Editing a pending submission creates a new `Email` revision (content-addressed) and re-points the submission; the old `Email` row is GC-eligible if unreferenced.

Where the state machine lives

Core defines the states, transitions, and invariants. *Driving* the machine is split:

- **Scheduler** (core or plugin) — moves `pending` → `queued` at `sendAt`.
- **Outbound channel plugin** (SMTP, Gmail API, …) — drives `queued` → `sending` → `sent`, and reports `failed`.
- **Delivery observer plugin** — watches for DSNs / provider callbacks, drives `sent` → `delivered`/`bounced`.

The engine enforces the legal-transitions table. A plugin attempting an illegal transition gets a wire-protocol error.

EmailObject: flags, not states

For received mail, `EmailObject` carries:

- `mailboxIds: Set<MailboxId>` — multi-membership.
- `keywords: Set<String>` — `$seen`, `$flagged`, `$answered`, `$draft`, `$forwarded`, plus user/plugin custom keywords (namespaced).
- `ext: { ... }` — per-plugin annotations (UI banner dismissed, AI classification, …).

Anything a UI or agent wants to model as "states" (e.g. a triage workflow: inbox → triaged → snoozed → done) is a *plugin* concern: that plugin defines its own keyword vocabulary or its own `ext` namespace with its own state machine. The core stays out of it.

Worked examples

- **User drafts an email, sends immediately.** Create `Email` (draft, `$draft` keyword, Drafts mailbox). On Send: create `EmailSubmission` (`pending`, `sendAt=now`); scheduler flips to `queued` immediately; SMTP plugin drives to `sent`; bounce-watcher drives to `delivered`. When `sent` is

reached, the `Email` gets the `$sent` keyword and moves from Drafts to Sent mailbox (a routing concern, not a state).
- **User schedules for tomorrow.** Same, but `sendAt=tomorrow`. The `EmailSubmission` sits in `pending` until then. User can edit (mutates `Email` + maybe `sendAt`) or cancel (`EmailSubmission` → `cancelled`, `Email` stays as draft).
- **Incoming mail arrives.** IMAP plugin fetches, calls `email.receive`. Engine creates (or dedups to existing) `Email`, creates `EmailObject` for the receiving account with `mailboxIds={Inbox}`, no keywords. No `EmailSubmission` involved. Emits `email.received` webhook for agents/UIs.
- **Reply.** New `Email` (with `In-Reply-To` + `References`), threading plugin attaches it to the existing `Thread`, new `EmailSubmission` drives send. On `sent`, the *original* `EmailObject` gains the `$answered` keyword (a side-effect declared by the reply submission).

Relationship to JMAP

One sentence

JMAP is our north-star data model and our reference client protocol; it is not our plugin protocol, and we extend it with `Channel` and `Address`.

Where we are JMAP-compatible

- Object shapes (`Email`, `Mailbox`, `Thread`, `EmailSubmission`, `Identity`) — superset. Our entity names deliberately match JMAP's.
- Multi-membership mailboxes.
- Immutable-artifact + mutable-state split (we make it explicit as `Email` vs. `EmailObject`; JMAP keeps it implicit inside `Email`).
- `EmailSubmission` lifecycle — our states are a superset of JMAP's.
- The idea of a *state token* for incremental sync.

Where we diverge

- **Plugin protocol ≠ client protocol.** JMAP is what *clients* speak to the engine. The *plugin* wire protocol is our own JSON-schema'd HTTP+webhook surface, which is broader (plugins can write metadata namespaces, register routes, emit events — none of which JMAP exposes).
- **Channel and Address are not in JMAP.** JMAP assumes one provider; we multiplex many.
- **Email / EmailObject split is explicit.** JMAP keeps both inside one `Email` object; we model them as separate rows to make dedup, encryption-at-rest, and per-account state ergonomic.
- **Metadata namespaces.** JMAP has no general extension mechanism beyond vendor-prefixed properties. We make `ext.<namespace>` a first-class, declared, version-checked surface.
- **Provenance is required.** JMAP doesn't track it.
- **No `/jmap/api` HTTP shape required of plugins.** Plugins speak the hackermail wire protocol; a JMAP-server plugin can *expose* JMAP to external clients on top.

Why not just be a JMAP server

We could. But:

1. JMAP doesn't model multi-provider muxing, which is the whole point.
2. JMAP doesn't have an extension story for arbitrary plugins (UIs, agents, custom protocols).
3. Forcing IMAP/SMTP/Gmail adapters to round-trip through JMAP wire format internally adds latency for no benefit.

So we *shape ourselves like JMAP*, ship a JMAP-server plugin for compatibility, and keep our plugin protocol free to evolve.

API Surface

This section sketches the wire protocol concretely enough to argue about. Schemas here are illustrative — the source of truth will live in `schemas/` as JSON Schema files; this section explains *shape* and *intent*.

Two surfaces, one envelope

There are two API surfaces:

- **Client API** — what UIs, agents, scripts, and external integrations call. This is the surface most readers think of as "the hackermail API".
- **Plugin API** — what the engine calls on plugins, and what plugins call back on the engine.

Both surfaces share: envelope shape, error model, type definitions, state tokens, capability tokens, and the JSON Schema registry. They differ only in which methods exist. A JMAP-server plugin is a thin adapter from JMAP to the client API; an IMAP plugin is a thicker adapter from IMAP to the plugin API.

Transport & envelope

Single HTTP endpoint per side: `POST /api` (client), `POST /plugin/api` (plugin). Batched JSON-RPC-flavored shape, lifted from JMAP:

```
{
  "using": ["hackermail.core@1", "hackermail.crypto@1"],
  "token": "opaque-capability-token",
  "calls": [
    ["Email/query", { "accountId": "a1", "filter": { "inMailbox": "INBOX" },
                      "sort": [{ "property": "receivedAt", "isAscending": false }],
                      "limit": 50 }, "c0"],
    ["Email/get",   { "accountId": "a1", "#ids": { "resultOf": "c0",
                      "name": "Email/query", "path": "/ids" } }, "c1"]
  ]
}
```

Response mirrors the shape:

```
{
  "sessionState": "2026-05-22T09:14:03Z/v42",
  "responses": [
    ["Email/query", { "accountId": "a1", "queryState": "q-9981",
                      "ids": ["e_AAA", "e_BBB", ...] }, "c0"],
    ["Email/get",   { "accountId": "a1", "state": "s-7720",
                      "list": [ /* Email objects */ ], "notFound": [] }, "c1"]
  ]
}
```

`#name` keys are backreferences to a prior call's response, resolved server-side. This is what makes "create-then-submit-then-mark-read" one network round-trip and one atomic transaction.

Method naming

`Type/Verb`. Verbs are JMAP-compatible where possible:

- `get` — fetch by ids.
- `query` — filter + sort, returns ids + `queryState`.
- `queryChanges` — delta since a `queryState`.
- `changes` — delta since a state token (id-level).
- `set` — create / update / destroy in one call.
- `copy` — cross-account or cross-type copy.

Plus a few we add:

- `view` — derive a transient view (e.g. decrypted body) without persisting.
- `subscribe` / `unsubscribe` — register a webhook for an event stream.

Client API — the surface

Method	Purpose
`Account/get`, `Account/changes`	List logical accounts the caller can see.
`Identity/get`, `Identity/set`	Sending identities (address, display name, signature).
`Mailbox/get`, `Mailbox/query`, `Mailbox/changes`, `Mailbox/set`	CRUD on mailboxes; supports both exclusive and inclusive semantics.
`Email/get`, `Email/query`, `Email/queryChanges`, `Email/changes`, `Email/set`	Immutable `Email` artifacts. `set` for upload/ import; mutation of body produces a new content-addressed row.
`EmailObject/get`, `EmailObject/set`, `EmailObject/changes`	Per-account mutable state: flags, mailbox membership, `ext` namespaces.
`Email/view`	On-demand derived view: decrypted body, rendered HTML, plaintext fallback. Never persisted.
`Thread/get`, `Thread/changes`	Read-only; thread membership is plugin-derived.
`EmailSubmission/get`, `EmailSubmission/set`, `EmailSubmission/changes`	Create / cancel / edit (only while `pending`). Drives the outbound state machine.
`SearchSnippet/get`	Highlighted snippets for a query, from the search plugin.
`Push/subscribe`, `Push/unsubscribe`	Register a webhook URL for event streams.
`Capability/list`	What capabilities the current token grants (introspection).

Search: a provider-agnostic filter algebra

`Email/query`'s `filter` is an *algebra*, not a fixed set of fields. Core defines the boolean structure (`AND`/ `OR`/`NOT`) and the always-available structured predicates served by the `metadata` role (`inMailbox`, `from`, `receivedAt`, `inThread`, keywords, ranges). Every *other* operator is contributed by an Index Provider (§Storage) that declared it via `index.analyzeSchema`:

- `text`, `phrase` — from an FTS provider.
- `semanticNear` — from a vector provider: `{ "model": "openai/text-embedding-3-small@1", "text": "lunch plans", "minScore": 0.78 }` (the engine routes `text → vector` through the embedding plugin, then runs ANN).
- future operators register the same way, no core change.

Two rules make this safe rather than a free-for-all:

- **Operators are capability-namespaced and declared in `using`.** A query using `semanticNear` must declare `using: ["hackermail.vector@1"]`; the engine rejects an undeclared or unsatisfiable operator at the envelope boundary — exactly like crypto methods. This keeps the wire surface and the conformance/golden-file tests well-defined even though operators are extensible. A query's meaning never depends on *which plugins happen to be loaded* — it depends on the declared `using` set.
- **Provider queries return ids WITH per-provider scores**, never a bare ranked list. A `text` result carries its BM25-ish score, a `semanticNear` result its similarity. Scores are what make ranking and fusion possible (§Search: fusion); without them a caller could only intersect, not rank.

Worked example — send an encrypted reply

One round-trip, four chained calls:

```
{
  "using": ["hackermail.core@1", "hackermail.crypto@1"],
  "token": "tok_...",
  "calls": [
    ["Email/set", {
      "accountId": "a1",
      "create": {
        "draft1": {
          "mailboxIds": { "mbx_drafts": true },
          "keywords":   { "$draft": true },
          "from":       [{ "email": "me@example.org" }],
          "to":         [{ "email": "alice@example.org" }],
          "inReplyTo":  ["<msg-1234@example.org>"],
          "subject":    "Re: lunch",
          "bodyText":   "yes — 13:00 works"
        }
      }
    }, "c0"],
    ["EmailSubmission/set", {
      "accountId": "a1",
      "create": {
        "sub1": {
          "emailId":     { "#ref": "c0/created/draft1/id" },
          "identityId":  "id_me",
          "sendAt":      "2026-05-22T11:00:00Z",
          "sign":        "pgp",
          "encrypt":     "pgp",
          "onSuccess": {
            "moveTo":    "mbx_sent",
            "addKeywords": ["$sent"],
            "removeKeywords": ["$draft"]
          }
        }
      }
    }, "c1"],
    ["EmailObject/set", {
```

```
      "accountId": "a1",
      "update": {
        "{originalEmailObjectId}": { "keywords/$answered": true }
      }
    }, "c2"]
  ]
}
```

Engine atomically: creates draft `Email`, creates `EmailSubmission` (which routes through the PGP crypto plugin to produce the encrypted wire `Email`), flags the original as answered. If any step fails, none of the writes commit.

Plugin API — engine calling plugins

The engine calls plugins for protocol work, storage, threading, etc. Same envelope. Method namespace is `<role>.<verb>`:

Method	Called on
`protocol.fetch`, `protocol.send`, `protocol.idle`	Protocol plugins (IMAP, SMTP, Gmail, …).
`storage.put`, `storage.get`, `storage.query`, `storage.delete`	Storage plugin.
`thread.derive`	Threading plugin: takes an `Email`, returns thread id.
`crypto.envelope`, `crypto.sign`, `crypto.encrypt`, `crypto.verify`, `crypto.decrypt`	Crypto plugins.
`keystore.sign`, `keystore.decrypt`, `keystore.listKeys`, `keystore.findKey`	Keystore plugins.
`route.resolveInbound`, `route.selectOutbound`	Routing plugin.
`index.put`, `index.query`, `index.delete`	Search/index plugin.
`agent.notify`, `ui.notify`	Agent / UI plugins (push, registered via webhook).

Plugin → engine callbacks

Plugins call back into the engine to push data and emit events:

Method	Purpose
`email.receive`	Submit a received RFC5322 blob; engine dedups, creates `Email` + `EmailObject`.
`email.materialize`	Submit a constructed `Email` (e.g. from a non-RFC5322 source like a bridge).
`emailSubmission.advance`	Move a submission to the next state with provenance.
`event.emit`	Emit an event onto the bus (`email.received`, `mailbox.changed`, etc.).
`metadata.write`	Write to a declared `ext` namespace on a core entity.
`schema.register`	Register/update a JSON Schema for an `ext` namespace at startup.

Event bus

Events are first-class. Subscribers register a webhook via `Push/subscribe`; the engine POSTs to it. Webhook payloads share the envelope.

Standard event types (extensible):

- `email.received` — new `Email` + `EmailObject` row created.
- `email.changed` — `Email` row mutated (rare — content-addressed).
- `emailObject.changed` — flags, mailbox set, or `ext` mutated.
- `emailSubmission.transitioned` — state machine moved.
- `mailbox.changed` — created / renamed / membership changed.
- `crypto.envelope.detected` — see §Crypto.
- `email.analyzed` — an `AnalyzedDocument` is ready for one `Email` (see below). This is the fan-out point for Index Providers.
- `account.*`, `identity.*`, `thread.*` — analogous.

Events carry `{ type, accountId, ids, stateBefore, stateAfter, at, provenance }`. Subscribers can filter at subscribe time (`{ types, accountIds, mailboxIds }`).

Analyze once, index many

With more than one derived index (FTS *and* vectors *and* …), the naive design has each Index Provider independently re-parse MIME, strip HTML, decrypt, and extract attachment text — the expensive work, redone per provider. Instead we factor the inbound enrichment into two stages, the standard analysis→indexing split:

1. **Analysis (once per `Email`).** A single analyzer subscriber turns raw content into an `AnalyzedDocument` and emits `email.analyzed`.
2. **Indexing (per provider).** Index Providers subscribe to `email.analyzed` — *not* raw `email.received` — and each builds its own structure from the shared document. Parse/extract/decrypt happens exactly once; adding a provider adds a subscriber, not another extraction pass.

```
email.received (committed)
  └─▶ analyzer ──produces──▶ AnalyzedDocument ──emits──▶ email.analyzed
        ├─▶ FTS provider     (tokenize → inverted index)
        ├─▶ vector provider  (chunk → embed → ANN index)
        └─▶ facet provider   (extract fields → structured index)
```

Analysis runs *post-commit* (it is expensive and must not block the inbound transaction), as the first subscriber after `email.received`.

Two hook tiers: interceptors vs. subscribers

The event bus above is *post-commit* and *fire-and-forget*: by the time `email.received` fires, the `Email` and `EmailObject` already exist. That is right for notify / index / AI-classify, but it cannot *shape* an ingest — it can only react to one. We need a second, distinct tier.

1. **Interceptors** — *synchronous, ordered, pre-commit.* They run *inside* the `email.receive` lifecycle, on a RAM-only artifact, and return a decision the engine acts on before any write commits. Allowed verdicts:
   - `transform` — rewrite/normalize the artifact (e.g. strip a tracking pixel, canonicalize MIME) before it is stored.
   - `annotate` — attach an `ext.<namespace>` record that lands atomically with the row (e.g. a synchronous classification).
   - `decide-retention` — a *retention directive*, not a bit: the hook that lets a forgettable sink (§Deployment Profiles) decide what to keep *before* writing, instead of store-then-evict.
2. **Subscribers** — *asynchronous, post-commit, at-least-once.* The existing event bus. They observe committed state and cannot veto it.

We borrow the *shape* from Stalwart's MTA-Hooks (sync HTTP POST returning an action — §Prior Art) but strip the MTA verbs: as a client-as-server we receive *already-accepted* mail from a provider,

so there is no `reject/quarantine/bounce` — we cannot un-receive what the provider already holds. The meaningful pre-commit decision is not "refuse" but "transform / annotate / decide-retention".

Retention is a directive, not a bit

"Store or not" is too coarse. The cheap, always-plaintext part (headers) and the large, often-encrypted part (body + attachments) deserve separate decisions, so `decide-retention` returns a directive over a lattice:

```
retention {
  metadata: keep | drop      // EmailObject + headers (small, plaintext, queryable)
  blob:     keep | drop      // raw body + attachments (large, often encrypted)
  ttl:      duration | null  // forget after N — the per-message forgettable knob
  reason:   string           // auditable cause; required when anything is dropped
}
```

Gradient: `full` ▶ `headers-only` ▶ `ephemeral` ▶ `drop`. "Headers-only" is the common privacy/compliance shape — keep enough to thread, search, and show "mail from X about Y" without retaining content.

- **Decision inputs are cheap by default.** Interceptors decide on envelope, headers, size, and provenance — all plaintext, no body materialization. A filter that needs body content is a *heavier* interceptor that must materialize/decrypt and is therefore capability-gated like privileged search (§Crypto).
- **Composition: most-restrictive-wins, under a profile ceiling.** Across the interceptor chain the engine takes the lattice *minimum* — any interceptor may downgrade retention, none may upgrade past the deployment profile's ceiling (§Deployment Profiles). A privacy `drop` cannot be overridden by a "keep everything" archiver.
- **`drop` ≠ "never happened".** A dropped artifact still flows through the RAM-only pipe — interceptors run, a synchronous notify can fire — it simply never lands in durable storage. (Post-commit subscribers, by definition, do not see a never-committed artifact; "process-then-forget" notification must be a *synchronous* notify in the interceptor chain.)
- **Blob retention is refcounted; metadata retention is per-account.** Because `Email`/blobs are content-shared, `blob: drop` for one account must not destroy another account's copy — the blob goes only when the last referencing account drops it (the `expunge` vs `destroyEmail` split, §Verbs).
- **Never drop silently.** An affirmative drop writes a body-less **tombstone** (`{at, channel, fingerprint, droppedBy, retention, reason, cause: "policy.drop"}`) so "where did my mail go?" is answerable without retaining content. This is §Provenance, inverted.

Unavailable is not "drop" (the durability rule for interceptors)

The single most dangerous conflation: a retention interceptor being *down* is not the same as it deciding `drop`. Discard happens *only* on an affirmative verdict — never because a plugin failed to answer. So the "fail open vs. fail closed" choice has three outcomes, and the broken one is forbidden:

- **fail-open** — provisionally `keep` and commit now; reconcile when the plugin returns. Never loses mail; may briefly store something policy would have dropped.
- **fail-closed (defer)** — hold the ingest *uncommitted*, do not advance the provider sync cursor, retry with backoff (§Durability & Crash Recovery). Never loses mail; ingest stalls while the plugin is down.
- **fail-by-dropping** — *forbidden.* Absence of a decision must never discard a message.

Discipline (so interceptors stay safe):

- Interceptors are ordered and declared in config (auditable). The per-interceptor failure policy (`fail-open` | `defer`) is declared, and defaults to whichever the deployment profile sets — never to discard.

- Only interceptors may block the inbound lifecycle. Everything optional (indexing, AI, UI notify) stays a subscriber and runs after commit, per the "never block the inbound path on optional plugins" rule (§Plugin timeouts & degraded mode).
- Subscriber delivery declares a drop policy — `lossless` (queue/retry to the DLQ) or `lossy` (drop under backpressure) — so a slow UI consumer cannot stall the bus. (Adapted from Stalwart's lossy/lossless subscriber channels.) Note this `lossy` is a *subscriber* choice on committed state; it never applies to the inbound retention decision.

State tokens & sync

Every typed result carries a `state` field; clients pass it back to `changes` / `queryChanges` to receive deltas. State tokens are opaque strings (clients must not parse them). A client that never sees push can poll `changes`; a client that subscribes to push uses tokens for catch-up after disconnect. Webhooks are best-effort; tokens are authoritative.

This is the engine's *upward* sync boundary (engine → UI clients / agents). As a client-as-server, keeping clients coherent is the engine's core job, so the mechanism is worth pinning down concretely rather than leaving as "opaque string". We adopt the shape proven by Stalwart (see §Prior Art):

- **Change-id.** A monotonically increasing `u64` counter per `(account, collection)`. Every mutation bumps it; it never moves backward. A `state` token is just the current value of this counter, rendered opaque to the client.
- **Changelog.** An append-only log of change records keyed by `(accountId, collection, changeId)`, so a `changes` call is a range scan from the client's last-seen id forward — not a re-scan of the whole collection. Storage plugins expose this as a first-class subspace (see §Storage); a forgettable store keeps only a bounded tail of it (see §Deployment Profiles).
- A change record carries `{ changeId, collection, entityId, op: created|updated|destroyed, provenance }`. Provenance is the same structured record required everywhere else (§Provenance) — the changelog is *why-aware*, which is what makes a diverging sync debuggable.

The `changes` round-trip (worked)

1. Client fetches its inbox; engine returns the data plus `state="42"`. The client stashes `42`.
2. A *different* client (the user's phone) flags one email and archives another. The engine appends change records `43` and `44`, bumping the account's `EmailObject` counter to `44`.
3. The first client reconnects and calls `changes(since: "42")`. The engine range-scans the changelog from `43`, returns "these two `EmailObject`s changed" plus `state="44"`.
4. The client fetches *only those two* rows, never the whole inbox.

`assert_state` — optimistic concurrency

Writes carry the client's base state. An `EmailObject/set` says, in effect, "I am acting on state `44`". If a concurrent mutation already advanced the counter to `45`, the engine rejects the write with `stateMismatch` (§Error model) — "you are working from stale data, re-sync first" — instead of silently clobbering. This is the mechanism that *produces* the `stateMismatch` code: every `set` asserts the caller's base state at the envelope boundary before any write commits.

Container vs. item change streams

"Changed" is not one stream but two, tracked under separate change-ids, because the two kinds of state churn at wildly different rates:

- **Item changes** — individual `EmailObject` mutations (flag, move). High volume, constant.
- **Container changes** — `Mailbox` create / rename / membership. Rare.

Splitting them lets a client subscribe to item changes without being woken for every mailbox rename, and lets the engine serve a cheap "anything structural changed?" check. This split is not incidental: it lands exactly on our `Email` (immutable, content-addressed, almost never changes) vs. `EmailObject`

(per-account mutable state, churns constantly) division from §Domain Model. The high-churn item stream *is* `EmailObject`; `Email` rows barely move. The seam we drew for storage and crypto reasons turns out to be the right seam for sync too.

Two boundaries, not one

A full mail server (an MTA) is the *source of truth* and has a single sync boundary: push changes up to clients. We are a client-as-server — a projection of *other* sources of truth (the providers) — so we have *two* boundaries, and they are asymmetric:

- **Upward** (engine → UI clients): identical to an MTA's only boundary. The change-id machinery above covers it.
- **Downward** (provider → engine): reconciliation against a truth we do *not* control — UIDVALIDITY rolls, Gmail `historyId` gaps, provider re-keying. There is no clean monotonic counter handed to us here; we reconstruct order from provider-specific anchors. This is the harder problem, and it is *ours alone* — it is what §Provenance, the channel op log (§rule 4), and the `uidValidityRolled` / `syncTokenExpired` error codes exist to handle. We do not paper the two boundaries together: the upward token is authoritative for clients; the downward state is forever a best-effort mirror of the provider.

Capability tokens

Every request carries a `token` field. Today: opaque string, accepted, logged. Tomorrow: real check. The token's claims include:

- `account` — which Accounts the holder may touch.
- `scope` — set of capability strings (`email.read`, `email.send`, `email.decrypt`, `metadata.write:ns=...`, `mailbox.admin`, …).
- `exp`, `iss`, `sub` — JWT-ish, or NATS-style nkey signed claims — decision deferred to Phase 5.

Methods declare required capabilities in their schema; the engine enforces (one day) or logs (today) at the envelope boundary, before any plugin sees the call.

Error model

Errors are values, not exceptions on the wire. Each method response slot is either a success body or:

```
{ "type": "error", "code": "invalidArguments",
  "description": "missing field: emailId",
  "details": { "path": "/calls/1/args/emailId" } }
```

Standard codes: `unauthorized`, `forbidden`, `notFound`, `stateMismatch`, `invalidArguments`, `overQuota`, `serverFail`, `pluginUnavailable`, `pluginTimeout`, `rateLimited`, `authExpired`, `providerUnavailable`, `syncTokenExpired`, `quotaExceeded`, `uidValidityRolled`, `conflict`. New codes require an RFC.

The latter group exists because the engine treats them very differently from generic failures: `rateLimited` triggers backoff, `authExpired` triggers a re-auth flow, `syncTokenExpired` / `uidValidityRolled` trigger sync repair, `conflict` surfaces a divergence between local and provider state.

Atomicity: within a single request, `set` calls are transactional per type. Cross-type atomicity is achieved by ordering calls and using backreferences; if a later call fails, the engine rolls back earlier `set`s in the same request (configurable via `"onError": "rollback" | "continue"`).

Versioning

- The `using` array declares which capability+version sets the caller speaks. The engine rejects methods outside what's declared.
- Schema evolution: additive within a major version. Breaking changes bump the major and require co-existence (engine speaks both for one release).

- Plugins declare which versions they target in their manifest. Engine refuses to load a plugin whose declared versions are unsatisfied.

What's deliberately *not* in the API

- Authentication / login. Token issuance is an authz-plugin concern; the engine consumes tokens, it does not mint them.
- File upload as a special endpoint. Attachments are part of an `Email`'s body parts; large blobs use a separate upload URL returned by `Email/set` (JMAP's blob pattern).
- HTML rendering, image proxy, link-tracking-strip — all client-plugin concerns.
- Bulk import as a magic verb. Bulk = a batched `Email/set` with N creates; the engine handles it the same way as any other batch.

End-to-End Crypto (S/MIME & OpenPGP)

Thesis

We do not implement crypto. We design the plumbing so that S/MIME and OpenPGP plugins can live inside our model without leaking plaintext into places it doesn't belong, and without forcing every other plugin (UIs, agents, storage, search) to understand crypto primitives.

This section is long because the failure modes are subtle: a naively designed system leaks plaintext into logs, swap, the search index, the sent folder, replies, and debug dumps. We want the abstractions that make those leaks *impossible by construction*, not "policed by code review".

Goals

1. Support PGP/MIME, inline-PGP, S/MIME (signed + enveloped), in any combination, on inbound and outbound.
2. Private key material lives in one place and never crosses plugin boundaries.
3. Plaintext lives only where it must, never on disk in canonical storage, never in logs.
4. Signature verification status is a first-class, queryable property of every received `Email` — without other plugins having to parse MIME.
5. UIs and agents render verification badges by reading a namespace; they do not call into crypto code.
6. Search of encrypted bodies is opt-in, isolated, and survives without it.

Non-goals

- A keyserver / WKD implementation in core (lives in the crypto plugin).
- Webmail-style in-browser key generation (a UI-plugin concern).
- Hiding the existence of crypto from the core: provenance and MIME awareness *are* core responsibilities.

Two plugins, not one

We split the responsibility into two plugin roles. Either may be implemented multiple times (one per scheme):

1. **Keystore plugin** — owns key material. Exposes operations: `sign(blob, keyId)`, `decrypt(blob, keyId)`, `listKeys(account)`, `findKeyForRecipient(address)`. Private keys *never* leave this plugin's process. Implementations: gpg-agent bridge, OS keychain, a PKCS11 / HSM bridge, a software keystore with passphrase, a hardware token (YubiKey).
2. **Crypto plugin** — owns MIME-level crypto: parse encrypted/signed envelopes, drive verification, produce encrypted/signed outbound envelopes. *Calls* the keystore for primitives; never holds keys itself. Implementations: `hackermail.pgp`, `hackermail.smime`.

The split is the entire point: a keystore can be reused across schemes, a scheme can be replaced without touching key material, and the "capability to decrypt for account X" becomes a single, auditable permission — the right shape for our future authz model.

Canonical storage is always the wire form

`Email.raw` is the RFC5322 bytes *as transmitted* — encrypted/signed if that's what crossed the wire. We do *not* store decrypted plaintext as canonical state. Reasons:

- Content-addressing matches what other systems saw.
- Backups, replication, and migration handle one artifact, not two.
- Lost keys mean lost content — which is the correct failure mode for end-to-end crypto; pretending otherwise is the leak.

Decrypted views are *derived, transient, and never persisted* except into:
- a clearly-marked privileged search index (opt-in, §Search below), or
- a UI's in-memory render (caller's responsibility, not ours).

Inbound flow

1. Channel plugin (IMAP, Gmail, …) calls `email.receive` with raw RFC5322.
2. Engine creates `Email` row (content-addressed) + `EmailObject` for receiving account.
3. Engine inspects MIME top-level type. If it matches a registered crypto envelope (`multipart/encrypted`, `multipart/signed`,`application/pkcs7-mime`), engine emits `crypto.envelope.detected` with the `Email` id and envelope type.
4. The relevant crypto plugin handles the event:
   - For *signed*: verifies via keystore lookup of signer's public key material (Autocrypt cache, WKD, keyring, CA chain for S/MIME). Writes a `crypto` namespace record on the `EmailObject` with `{ signed: true, verified: bool, signer, signedAt, trustPath, errors[] }`.
   - For *encrypted*: does *not* decrypt eagerly. Just records `{ encrypted: true, recipients[], canDecrypt: bool }` (the latter by asking the keystore which recipient keys are available).
5. Threading, indexing, and UI plugins proceed using headers (which are always plaintext) and the `crypto` namespace. They do not see plaintext bodies.

On-demand decryption

When a UI or agent needs the plaintext body, it calls a wire-protocol method like `email.view(email_id, { decrypt: true })`. The engine routes the call to the crypto plugin, which:

1. Checks the caller's capability (today: stub; tomorrow: `email.decrypt:account=X`).
2. Asks the keystore to unwrap the session key.
3. Decrypts the body, returns a *transient* `Email`-shaped view (parsed MIME tree, headers from the outer envelope merged with protected headers if present).

The transient view is *not* stored. The engine may cache it in-memory for the duration of a sync session, behind a TTL and a "plaintext-allowed" capability check.

Outbound flow

Composition stays plaintext-aware: the UI plugin produces a draft `Email` in cleartext (so the user can edit it, the search index can see it, attachments can be added). On `EmailSubmission`, the user (or UI policy, or autocrypt heuristics) declares intent:

```
EmailSubmission {
  emailId: "...",
  sign:    "pgp" | "smime" | null,
  encrypt: "pgp" | "smime" | null,
  ...
}
```

The submission pipeline:
1. Engine asks the chosen crypto plugin to produce the wire form.

2. Crypto plugin asks the keystore for the sender's private key (sign) and recipients' public keys (encrypt), per-recipient.
3. Crypto plugin emits the wire `Email` (a *new* content-addressed row).
4. The draft `Email` (cleartext) is *detached* from the submission and either:
   - retained encrypted-to-self (default — see below), or
   - deleted (`policy=ephemeral`), or
   - kept as a local-only draft (`policy=local-plaintext`, dangerous — requires explicit user setting).
5. SMTP / Gmail plugin transmits the wire form.

Encrypt-to-self (default)

For Sent folder UX and multi-device sync, the crypto plugin adds the sender's own encryption key as an additional recipient. The same wire ciphertext now contains a key packet that the sender can later unwrap — the user can read their own Sent mail on another device, without us having stored plaintext anywhere.

This is what real PGP clients do. We make it the default; surfacing the trade-off ("you cannot read your own sent mail if you lose this key") is a UI concern.

The crypto namespace on EmailObject

The crypto plugin writes a stable, declared schema under `ext.hackermail.crypto`. Every UI and agent reads this — they never parse MIME themselves.

```
{
  "scheme": "pgp" | "smime" | null,
  "signed":    { "verified": true, "signer": "...", "signedAt": "...",
                 "trustPath": [...], "errors": [] } | null,
  "encrypted": { "recipients": ["..."], "canDecrypt": true,
                 "decryptedOnce": false } | null,
  "warnings": ["mixed-content", "protected-headers-mismatch", ...]
}
```

A UI badge ("verified by `alice@example.com`", "decrypt failed") is a direct render of this namespace.

Protected headers & Autocrypt

- **Protected headers** (RFC 9078 / Autocrypt level 1): some headers (Subject, From) are duplicated inside the encrypted body so they're authenticated. The crypto plugin surfaces *both* outer and protected values; the engine prefers protected for display when verification passes, and flags `protected-headers-mismatch` otherwise.
- **Autocrypt**: opportunistic public-key advertisement via `Autocrypt:` header. The crypto plugin maintains its own namespace cache of seen keys per sender. This is a plugin-local concern — no core changes.

Replies & forwards of encrypted mail

A reply that quotes encrypted content must compose with the *decrypted* quote in the draft body. Mechanics:

1. UI plugin requests decrypted view from crypto plugin (capability check).
2. UI inserts quoted text into new draft `Email` (cleartext, as usual).
3. User chooses whether to encrypt the reply (UI default: match the thread's prior encryption).

The draft is plaintext like any draft. The decision to encrypt the outbound reply is at `EmailSubmission` time, not before.

Search of encrypted bodies

Default: encrypted bodies are not indexed. Header-level and metadata-level search continue to work (headers are plaintext, the `crypto` namespace is searchable).

Opt-in: a *privileged search plugin* may declare the capability `email.decrypt:account=X` (today: stub-trusted; tomorrow: real enforcement). It then:

1. Subscribes to `email.received`.
2. For each encrypted `Email` matching the capability, requests the decrypted view, indexes the plaintext.
3. Stores the index in its own datastore, ideally encrypted at rest with a key from the keystore (closes the loop — we don't reintroduce the leak we just avoided).

We ship this plugin separately and off-by-default. Operators who want it opt in knowingly.

Hard problems & where they live

- **Key discovery & trust UX.** WKD lookup, keyserver fetch, Autocrypt state machine, S/MIME CA store — all crypto-plugin concerns. Core exposes no opinion.
- **Key rotation.** Old mail must remain decryptable with old keys; the keystore plugin owns historical key material.
- **Mixed signed + encrypted nesting.** PGP allows arbitrary nesting; the crypto plugin recursively descends and produces a single normalized `crypto` namespace record.
- **Malformed crypto envelopes.** Treated as soft failures: `Email` is stored as-is, `crypto.warnings` is populated, body is not silently shown as if it were plaintext.
- **Side-channel hygiene.** Plaintext must never enter logs, traces, or error messages. The engine's logger has a "plaintext-tainted" flag on values returned from crypto-plugin decrypt calls; tainted values are redacted by default. (Mechanism is core; enforcement is engine-wide.)
- **Time-of-receipt verification vs. time-of-display verification.** Verification at receipt freezes trust state; trust can change later (revoked cert, expired key). We store both: the receipt-time verdict and a re-verify hook. UIs display the receipt-time verdict, with a badge if a re-verify diverges.

What this means for the core

Almost nothing changes in the core. We add:

- A registry for MIME types that trigger crypto envelope events.
- A "plaintext-tainted" marker in the value type used across the wire protocol (so the logger can redact).
- An on-demand `email.view` method that may be routed through a crypto plugin.
- Reserved namespace key `ext.hackermail.crypto` with a versioned schema.

That is the entire core surface area for end-to-end crypto. Everything else lives in the two plugin roles.

Storage & Query Layer

Storage is plugged, not fixed

Core defines the storage *interface* (CRUD + search + event stream) over the canonical model above. Reference implementation: SQLite for state + content-addressed blob store for `Email` bodies. Postgres, S3, FoundationDB, a CRDT store, or even IMAP-as-backing-store must all be possible implementations.

Four storage roles, not one

"Storage is a plugin" is too coarse. Storage is really *four* roles with different durability, latency, and size profiles, each independently swappable (a pattern Stalwart proves in production — see §Prior Art). Splitting them is what lets "encrypted bodies in S3, state in SQLite, index in Elasticsearch, hot cache in Redis" fall out without special cases — and, crucially for us, what lets "forgettable" *compose* rather than being a fifth monolithic store.

Role	Holds	Forgettable variant
`metadata`	`EmailObject`, `Mailbox`, flags, `ext` namespaces, the changelog.	in-memory, TTL-evicted
`blob`	Immutable `Email.raw` + attachments, content-addressed.	null (drop after fanout) / TTL
`index`	*Index Providers*: inverted FTS, vector/ ANN, faceted, … (see below).	disabled (no index)
`cache`	Hot derived state, decrypted-view TTL cache, sessions.	in-memory (already ephemeral)

Content addressing is by hash of `Email.raw`; we specify SHA-256 in §Identity for `rawHash`, but for the blob-store *key* a faster 256-bit hash (e.g. BLAKE3) is equally valid — it is an address, not a signature. Implementations may use either as long as `rawHash` itself stays SHA-256 for the cross-system identity guarantee.

The changelog is a storage role concern

The sync changelog (§State tokens & sync) is a first-class, range-scannable partition of the `metadata` role — keyed `(accountId, collection, changeId)`. A storage plugin must expose it as such; a forgettable store keeps only a bounded tail (the retention window), which is what couples the TTL to the idempotency and replay horizons (§Deployment Profiles).

Index Providers (one role, many instances)

Full-text search and AI embeddings are not two features — they are two *instances* of one shape: a derived, queryable projection of email content that is not canonical state. Rather than special-case each, the `index` role is an **Index Provider** contract, and FTS, vector/ANN, faceted, language-detect, near-duplicate clustering, etc. all plug in as instances with no core changes. The contract is small:

```
index.analyzeSchema()       → declares the filter operators + sort keys
                              this provider can satisfy
index.upsert(analyzedDoc)   → build/refresh structure for one Email
index.delete(emailId)       → cascade on expunge/forget
index.query(subFilter)      → return ids WITH per-provider scores
```

A provider *declares the operators it satisfies* (`text`, `phrase`, `semanticNear`, `hasAttachment`, `lang`, range, …). That declaration is how the engine routes queries without hardcoding knowledge of FTS or vector — and the operators are capability-namespaced (§API Surface), so they are gated by `using` just like crypto methods.

Why this is not an `ext` namespace: a derived index is large and exists to be queried by its *own* structure (an inverted index, an HNSW/IVF graph), not returned inline with `Email/get`. `ext` blobs ride along with every read; index structures must not.

Vector/embedding providers, specifically

The embedding case is just a vector Index Provider. Two model-specific rules it adds:

- **Multiple embeddings per email, keyed `(model, version, scope, chunkId)`.** Vectors from different models occupy different geometric spaces and are not comparable, so a query targets exactly one `(model, scope)` space; long emails chunk into many vectors (RAG); model upgrades version-bump and backfill before GC of the old space. One `Email` therefore carries many embeddings by design.
- **Content-shared, account-scoped.** The vector is a pure function of `(content projection, model, version)`, so it is content-addressed and computed once per `Email`, shared across every

account that received the message — while *index membership* stays per-account. Identical to how FTS body-tokenization is content-derived but the search index is per-account. A small descriptor under `ext.hackermail.embedding` (`[{model, dim, version, scope, vectorRef}]`) lets UIs/agents discover *which* embeddings exist without moving vectors; the vectors live in the index role, referenced by id (exactly how attachments reference blobs).

One lifecycle, defined once on the role

All Index Providers share an operational lifecycle, specified once so no instance reinvents it:

- **Backfill** on registration (new provider walks the corpus).
- **Incremental** update on `email.analyzed` / `emailObject.changed`.
- **Deletion cascade** on `expunge` / forget.
- **Reindex** on analyzer/model version bump (version-in-key, dual-read during backfill).
- **Capability gate** for encrypted content (§Crypto) — one rule, every provider.
- **Forgettable profile** — provider holds its structure in its *own* external store; the engine retains nothing.
- **Degraded mode** — a provider that is down returns `pluginUnavailable` for its operators; the planner drops or fails those predicates per policy while other predicates still resolve (the "never block on optional plugins" rule, applied to search).

Header-level and metadata-level filters are always served by the `metadata` role's own secondary indexes and work even when bodies are encrypted or unindexed; Index Providers are *additional, optional* projections layered on top.

Sync model

We adopt JMAP's *state token* pattern: every queryable type exposes a monotonic state token; clients and plugins pull deltas (mechanism in §State tokens & sync). Webhooks complement this for push, but the state token is the source of truth for "did I miss anything".

Routing & Multiplexing

Enterprise use cases (catch-all, split send/receive, per-address routing, mass-account handling, one domain → many accounts) are *routing plugin* concerns, not core concerns.

Core provides:

- `Address → Account` resolution hook.
- `Account → Channel(s)` selection hook for outbound.
- `Channel → Account(s)` fan-in hook for inbound.

A default routing plugin ships with sensible 1:1 behavior. Enterprises write or install a routing plugin that handles their topology.

Authorization (seam-only today)

Today

No real authz. Single trusted operator. All plugins are trusted.

The seam

Every wire-protocol call carries a `token` field (opaque string today, ignored). Every plugin manifest declares `capabilities` it claims. The engine logs but does not enforce.

Tomorrow (10-year shape)

- NATS-style decentralized tokens, or a capability mapping table.
- Capabilities are fine-grained: `mailbox.read:account=X`, `metadata.write:ns=my.plugin`, `email.send:from=@domain`.
- Tokens are issued by an authz plugin (yes, also a plugin), verified by core on every call.

- Today's stub: token is accepted, capabilities are logged. Tomorrow: same call sites, real enforcement.

The point: *no call site changes* when authz lands. The token argument and capability declaration already exist.

Configuration

Hybrid, with code as truth

- **Config files (TOML/JSON)** — declarative baseline: which plugins, which endpoints, which accounts.
- **API** — runtime mutation; everything settable in config is also settable via API.
- **Code** — for power users, a Rust/TS embedding surface to define plugins and routing inline.

Decision rule

If a setting affects routing or security, it must be expressible in config (auditable). If it affects behavior, it must also be expressible via API (automatable). Code is a convenience layer over the API.

Deployment Profiles

Why profiles exist

Useful deployments — a receive-only privacy mail-drop, a webhook bridge, a phishing-analysis sink — are today only *emergent*: you assemble them from three independent decisions (which channel plugins load, which storage variant, which token scopes) and nothing checks they are consistent. That is exactly the failure the §Configuration "security settings must be auditable" rule warns against. A *profile* is a named, engine-asserted invariant over those decisions.

Stalwart (an MTA) reaches the same need via `ClusterRoles` — a struct of boolean capability flags each service checks before starting (§Prior Art). We adopt the *shape* (a checked capability declaration) at the scale of a client-as-server: a profile is a small config block the engine validates against the loaded plugin set *at startup* and refuses to run if violated.

The receive-only, forgettable profile

```toml
[profile]
name        = "receive-only-ephemeral"
outbound    = false        # no EmailSubmission, no outbound channel plugin
storage     = "ephemeral"  # metadata=in-memory+TTL, blob=null, index=off
retention   = "60s"        # TTL for the forgettable metadata role
journal     = "durable"    # intake journal stays durable (default) — see below
```

On startup the engine asserts:

- *No outbound channel plugin is registered* (else refuse to start). This is the mechanism half — there is literally nothing to drive an `EmailSubmission` past `pending`.
- *No issued token carries `email.send` scope* (§Authorization). This is the policy half. "Receive-only" is enforced at *both* layers, and the profile is the single auditable place that asserts "this engine cannot send" — which no single setting otherwise states.
- *The storage role binding matches an ephemeral implementation*, so "forgettable" is a checked property, not an accident.

Forgettable applies to content, never to the intake journal

"Ephemeral storage" forgets message *content*; it must *not* forget the intake journal (§Durability & Crash Recovery), or the profile loses its crash/outage guarantee — a momentarily-down interceptor or a restart would lose mail. So the profile carries `journal` as a separate dial, independent of `storage`:

- `journal = durable` (default) — *perfect sync*: survives crash and plugin downtime, exactly-once notification, at the cost of durable content-free fingerprints of every message (incl. dropped ones).
- `journal = none` — *true zero retention*, RAM-everything, at the cost of at-most-once delivery (a crash may lose in-flight mail or double-fire).

The default is `durable`: a receive-only sink almost always wants "even if the server blips, we did not lose or double-process mail" more than it wants to forget that a message *existed*. Zero-retention is the knowing opt-in for the rare maximally-private case. The two are genuinely exclusive — the journal is what makes sync perfect, and it is itself the one (small, content-free) thing retained.

Forgettable storage changes three semantics — state them, don't hide them

A forgettable store quietly inverts guarantees the rest of the design assumes. The profile must pin the coupling rather than let it fail silently:

1. **Dedup collapses onto the TTL.** `rawHash`/`fingerprint` dedup (§Identity) and the idempotency window (§rule 1, default 7 days) both assume the store *remembers* past arrivals. With a 60s TTL, the retention window *is* the dedup/idempotency window. The invariant: `retention ≥ idempotencyWindow ≥ webhookMaxRetryHorizon`. Configure them inconsistently and at-least-once delivery produces duplicates that outlive the dedup memory.
2. **Sync/replay authority inverts.** `changes`, `queryChanges`, and `Push/replay` (§rule 6) need a durable changelog to replay *from*. A forgettable engine keeps only a bounded changelog tail, so it cannot honor catch-up beyond the retention window. In this profile the relationship flips: webhooks become *authoritative* and best-effort, the state-token catch-up model is bounded. Half the sync API (`changes`/`Push/replay` past the window) is inert — the profile must advertise this so clients do not assume durable replay.
3. **Annotation lifetime is bounded.** Metadata lives *on* the object (§Metadata blob), so evicting an `Email`/`EmailObject` evicts any `ext.<namespace>` a subscriber wrote (AI classification, UI flags). Therefore in this profile *subscribers must externalize anything they want to keep* — the engine is a transient pipe, not a store of record.

Forgetting deserves precise verbs

We split `delete` into precise verbs (§Verbs) because "delete means too many things". *Forgetting* earns the same scrutiny: it is either TTL eviction or never-stored, and either way it cascades from `Email` to its `EmailObject`s and their `ext` namespaces. A profile that forgets must say which, so "we forgot it" never silently means "we also dropped a subscriber's annotation it assumed was durable".

Other profiles (sketch)

The same machinery names other shapes: `read-only-cache` (sync down, serve UIs, never mutate provider state), `archive-sink` (durable store, no outbound, no UI), `full` (everything, the default). Profiles are additive; none change core code.

Roadmap

Phased plan. Each phase is shippable on its own and proves a specific hypothesis. Dates are aspirational; phases are sized in weeks of focused work, not calendar weeks.

Phase 0 — Skeleton (2–3 weeks)

Hypothesis: the canonical model + plugin protocol can be expressed without writing any real protocol plugin.

- Define core entities as JSON Schemas (`Email`, `EmailObject`, `Mailbox`, `Thread`, `EmailSubmission`, `Identity`, `Account`, `Channel`, `Address`).
- Define the wire protocol: method list, request/response envelopes, webhook envelope, error shape.

- Implement engine: schema registry, plugin registry, in-memory store, event bus, capability-seam (stub).
- Ship one mock protocol plugin and one mock storage plugin to prove end-to-end flow.
- CI: schema lint, golden-file wire tests.

Phase 1 — Real email in, real email out (4–6 weeks)

Hypothesis: an IMAP plugin and an SMTP plugin can drive the engine without the engine knowing what IMAP or SMTP is.

- IMAP fetch + IDLE plugin.
- SMTP submission plugin.
- SQLite + blob storage plugin (replaces in-memory).
- Threading plugin (JWZ algorithm).
- Minimal routing plugin (1:1 address → account).
- CLI client plugin (read + send) — proves "UI is a plugin".

Phase 2 — Multiplexing & the enterprise story (3–4 weeks)

Hypothesis: catch-all, split send/receive, and one-domain-many-accounts fall out of routing-as-plugin without core changes.

- Routing DSL in the routing plugin.
- Catch-all + alias expansion.
- Split send/receive across channels.
- Stress-test scenarios 1, 4 from §Stress-test pass.

Phase 3 — Modern providers (4–6 weeks)

Hypothesis: Gmail and Outlook map cleanly onto our model without contorting it.

- Gmail API plugin (OAuth, labels → inclusive mailboxes, push via watch+Pub/Sub or polling fallback).
- Microsoft Graph plugin.
- JMAP-server plugin (exposes our engine to external JMAP clients — closes the loop with the standard).

Phase 4 — Extensibility ergonomics (3–4 weeks)

Hypothesis: writing a new plugin is a weekend project for an external developer.

- Plugin SDKs: Rust + TypeScript.
- Plugin scaffolding CLI.
- Schema-diff tooling for metadata namespace versioning.
- Reference web UI plugin (annotations, custom flags).
- Stress-test scenarios 3, 6 pass.

Phase 5 — Production hardening (ongoing)

Hypothesis: this is usable as a daily driver.

- Encryption-at-rest, PGP plugin (scenario 2).
- Backpressure & flow control on webhooks.
- Hot reload of plugins.
- Migration tooling for metadata schema bumps.
- Real authorization: replace the seam with NATS-style tokens or a capability table. *No call-site changes required.*
- Observability: structured logs, metrics, distributed tracing across plugin hops.

Phase 6 — Beyond email (speculative)

Hypothesis: the model generalizes.

- Matrix-bridge plugin (rooms as Mailboxes — scenario 5).
- Calendar/task entities as plugin-defined core extensions?
- Federated multi-engine sync.

Cross-cutting tracks

Run in parallel with the phases above:

- **Docs.** Every phase ships with updated spec + plugin author guide.
- **Conformance suite.** A test harness any plugin can run against itself to prove it implements the protocol correctly. Grows with each phase.
- **RFC process.** Changes to core entities or wire protocol go through a lightweight RFC, recorded in `rfcs/` next to this document.

Durability & Crash Recovery

The interceptor model (§Two hook tiers) decides *before* persistence, on a RAM-only artifact. That is efficient and privacy-preserving, but it raises the sharpest correctness question in the design: *if a plugin or the engine is down momentarily, do we lose mail or diverge?* The answer must be no. This section states the invariant and the machinery that holds it.

The no-loss invariant

For every message a channel observes, exactly one of {committed, affirmatively-dropped-with-tombstone} eventually becomes durable — regardless of transient plugin or engine failure.

RAM-only-before-persistence is *safe* only because some durable copy exists elsewhere to replay from. What that copy is differs by channel, and the distinction is load-bearing.

Replayable vs. non-replayable channels

Channels declare which they are; it decides whether RAM-only ingest is safe or whether a durable intake spool is mandatory.

- **Replayable** (IMAP, Gmail, Graph — pull-based). The message still exists *at the provider* until we acknowledge it (advance the UID cursor / `historyId`, or delete). The provider *is* our write-ahead log; on crash we re-fetch. RAM-only ingest is correct here on one condition: **never advance the provider sync cursor until the ingest outcome is durable.** Bumping the cursor before the keep/drop decision is durable opens a crash window that loses the message.
- **Non-replayable** (SMTP-in, webhook bridge, Matrix event — push-based). The message arrives once; the source will not re-deliver. RAM-only ingest loses mail on crash. These channels *must* durably spool the raw bytes on intake, *before* interceptors run — the engine is the only durable copy.

General rule: *the engine never relinquishes the only durable copy of a message until its outcome is itself durable.*

The intake journal (always durable)

A small, always-durable journal keyed by the idempotency key already required of every inbound callback (§rule 1: `channel:mailbox:uidvalidity:uid`, `channel:historyId:gm_msgid`):

```
{ "idempotencyKey": "...",
  "provenance":     { /* channel, at, provider.* */ },
  "state":          "received" | "decided" | "finalized",
  "outcome":        "keep" | "drop" | null,
  "rawRef":         "<spool-ref>"   // present only for non-replayable channels
}
```

It holds *no body* for replayable channels (re-fetch recovers it) and the spooled raw for non-replayable ones. It is the inbound analog of the post-commit DLQ + `Push/replay` (§rule 6), which today guards only the *upward* (engine → subscriber) path; the journal guards the *downward* (provider → engine → decision) path that had no equivalent. It delivers three guarantees at once:

1. **Idempotent retry** — a re-delivered webhook hits an existing key, no duplicate row, no double-fired notification.
2. **Crash recovery** — on restart, scan for any non-`finalized` record, re-acquire the body (re-fetch for replayable / reload spool for non-replayable), re-run interceptors. `finalized` keys are skipped.
3. **Plugin-down survival** — a message whose interceptor could not run sits pending; it is *held*, not lost, and retried when the plugin returns (the `defer` outcome, §Two hook tiers).

The `email.receive` lifecycle

Persistence and the retention decision must not race. `email.receive` therefore has an explicit ordering:

```
record intake (durable id [+ spool if non-replayable])
   → run interceptors (RAM body)
      → keep:        commit Email/EmailObject  → finalize → advance cursor
      → drop:        write tombstone           → finalize → advance cursor
      → unavailable: leave pending (do NOT advance cursor) → retry
```

The provider cursor advances *only* on `finalize`. Everything between intake and finalize is recoverable from the journal plus the durable source.

Perfect sync vs. zero retention — the dial

The journal retains a content-free *fingerprint* of every message, including dropped ones; that is exactly what buys exactly-once and crash-safety. A maximally-private sink may not want even that. So the journal is a profile dial (§Deployment Profiles):

- `journal = durable` (default) — perfect sync: survives crash and plugin outage, exactly-once notifications. Cost: durable message *fingerprints* (never content).
- `journal = none` — true zero retention, RAM-everything. Cost: at-most-once; a crash may lose in-flight mail or double-fire on recovery.

You cannot have both perfect-sync and zero-retention — the journal is the thing that makes sync perfect, and it is itself a (small, content-free) retention. "Forgettable" applies to message *content*; the intake journal stays durable even when everything else is RAM-only. This is the precise form of "store message ids always".

Real-World Quirks & Defensive Invariants

The model is now expressive enough that the hard problems are no longer "can we represent this?" but "can the engine maintain clean invariants while messy provider behavior pours in?". This section enumerates the defensive rules the engine must enforce, organized by the priorities that bite first when things go wrong.

Idempotency (rule 1)

Every plugin → engine callback carries an `idempotencyKey`. The engine deduplicates within a configurable window (default 7 days).

Recommended key shapes:
- Inbound from IMAP: `channel_id:mailbox:uidvalidity:uid`.
- Inbound from Gmail API: `channel_id:historyId:gm_msgid`.
- State transitions: `submissionId:targetState:nonce`.
- Event emissions: `plugin:eventId` (plugin-chosen, must be stable).

Without this, retried webhooks create duplicate `EmailObject` rows and duplicate state transitions. The engine refuses callbacks without an idempotency key; the cost of producing one is the cost of being correct.

Provenance everywhere (rule 2)

Already required (§Provenance). The point reinforced here: provenance is *the* debugging tool when state diverges. Every mutation carries `{channel_id, plugin, at, provider.*, cause}`. The engine refuses writes without it. Logs and traces include it. A "trace view" UI is a Phase 1 deliverable, not a Phase 5 nice-to-have.

Strict vs. fuzzy identity (rule 3)

Already encoded as `rawHash` (strict) + `fingerprint` (soft) on `Email`. The rule: *the engine never silently merges*. Linking via `fingerprint` is observable; merging is a UI/agent decision at display time. This prevents the "we ate the user's mail" failure mode.

Provider operation log (rule 4)

Every channel plugin maintains an append-only log of provider operations attempted, in their own namespace:

```
ext.hackermail.imap.opLog: [
  { "at": "...", "op": "MOVE", "src": "INBOX", "dst": "Archive",
    "result": "ok", "uidNext": "55822" },
  { "at": "...", "op": "EXPUNGE", "result": "providerError",
    "code": "AUTHENTICATIONFAILED" }
]
```

This is *separate* from canonical provenance: provenance tells you why hackermail mutated state; the op log tells you what the channel plugin tried to do at the provider. When the two diverge, you have a sync bug, and you can see it.

Mailbox & delete semantics (rule 5)

Already encoded as the precise-verb set (§Verbs). The rule reinforced: `removeFromMailbox` / `archive` / `trash` / `expunge` / `destroyEmail` are not aliases. They are distinct operations with distinct provider mappings and distinct undo semantics. A plugin that collapses them is a buggy plugin.

Event retry & dead-letter (rule 6)

Webhook delivery is at-least-once. Subscribers must be idempotent (see rule 1). The engine maintains:

- Per-subscriber retry policy (exponential backoff, max attempts).
- A dead-letter queue for events that exceed retries.
- A `Push/replay(since: token)` method so a subscriber that's been down can catch up authoritatively (state tokens are source of truth; webhooks are convenience).

Inside the engine, the event bus must be backed by a durable log, not an in-process channel. Otherwise a crash mid-fanout loses events.

Clear delivery & sync state meanings (rule 7)

Already encoded by splitting `accepted` / `assumedDelivered` / `delivered` (§EmailSubmission states), and by `syncTokenExpired` / `uidValidityRolled` / `authExpired` as distinct error codes (§Error model). The rule: we never paper over uncertainty. A UI that shows "Sent" is showing `accepted`; a UI that shows "Delivered" has real evidence. Misrepresenting these is a bug.

Conflict resolution

When local and provider state diverge (user archived in Gmail UI *and* in hackermail, IMAP sync brings news after a local change), the engine applies per-field merge:

- **`keywords`** (set): union, with per-keyword provenance. Conflicts are rare because additions and removals carry timestamps.
- **`mailboxIds` with `inclusive` semantics** (set): union by default, conflict only on the same mailbox concurrently added+removed.

- **`mailboxIds` with `exclusive` semantics**: latest-`at` wins, loser recorded in `ext.hackermail.core.conflicts` for surface.
- **`ext.<namespace>`**: namespace owner decides (declared in manifest: `mergePolicy: "lww" | "manual" | "custom"`).

The engine never silently discards a conflicting write. Loser values land in the `conflicts` namespace; UIs may surface them; the dead- letter equivalent for data.

Plugin trust (MVP)

Even before real authz, plugins authenticate to the engine with a shared secret minted at registration time (`X-Plugin-Token` header). The engine refuses calls without it. Plugins receive webhooks signed with HMAC over the body + a per-subscriber secret. This is the floor; the capability model layers on top later without changing call sites.

Plugin timeouts & degraded mode

Every engine → plugin call has a declared timeout (in the plugin manifest). The engine:

- Fails fast on timeout with `pluginTimeout`.
- Tracks per-plugin health (rolling error rate, p99 latency).
- Sheds load from unhealthy plugins (circuit breaker).
- Degrades gracefully: if the search plugin is down, queries return `pluginUnavailable` on `searchSnippet` but other reads still work.
- Never blocks the inbound path on optional plugins (indexing, AI agents, UI notifications run after the `email.receive` transaction commits).

What we are *not* encoding today

These belong in design but not in core code yet. Listed so we remember:

- **Namespace governance** (overlapping `priority`/`category`/`status` across plugins). Convention, not mechanism. Revisit in Phase 4.
- **Calendar invites** (iCalendar workflow). Plugin namespace today; promotion to a richer object only if usage forces it.
- **Rich attachment lifecycle** (virus scan, text extraction, dedup across mails). Today: blob ref + content-id. Plugin namespace for derivations.
- **Cross-plugin distributed tracing**. We require trace ids on provenance now; full OpenTelemetry integration in Phase 5.

Prior Art: Stalwart

Why look at it

Stalwart (Rust, JMAP-first, multi-backend storage) is the closest shipping system to our north star, so it is our richest source of *validated* mechanism. The essential caveat: *Stalwart is a full mail server (an MTA); we are a client-as-server.* It accepts mail from the internet over SMTP, filters it, and *owns* the mailbox — it is the source of truth. We connect to existing providers (IMAP / Gmail / Graph), which remain the source of truth, and re-serve a unified projection to UIs and agents. So we adopt its *local-sync-engine + store* half and ignore its *MTA-pipeline* half.

What we adopt

- **Change-id sync machinery** → §State tokens & sync. Monotonic `u64` per `(account, collection)`, range-scannable changelog, `assert_state` optimistic concurrency, container-vs-item split. This is the most important borrow: sync *is* our core job.
- **Storage as separable roles** → §Four storage roles. Their `Store` / `BlobStore` / `SearchStore` / `InMemoryStore` separation, plus their `Ephemeral` store variant, is our metadata/blob/index/cache split and our forgettable tier.
- **Checked capability profiles** → §Deployment Profiles. Their `ClusterRoles` boolean flags, validated before a service starts, become our startup-asserted profile.

- **Synchronous decision hooks** → §Two hook tiers. Their MTA-Hooks (sync HTTP POST returning an action) become our interceptors, minus the reject/quarantine verbs we cannot honor.
- **Small wins:** `arc_swap`-style atomic Arc swap for hot-reloading the plugin/registry config (resolves the hot-reload open question — it is cheap, so: yes); a *pluggable* cluster pub/sub abstraction defaulting to in-process (resolves the event-bus open question — abstract it, default `None`, let operators swap in NATS/Kafka); BLAKE3 as a blob-key hash.

What we deliberately reject

- **Re-encrypt-at-rest on ingest.** Stalwart re-encrypts received cleartext to the account key at ingest. We do the opposite (§Crypto): canonical storage is the *wire form as transmitted*, never re-encrypted — to preserve content-addressing, provenance, and provider round-trips, and to treat lost-keys as the correct failure. "Encryption at rest" for us means wire-form preservation, *not* Stalwart-style re-encryption.
- **The MTA pipeline.** SMTP-stage hooks, SPF/DKIM/DMARC/ARC inbound auth, the outbound queue/DSN/retry spool, and the spam-filter stages are source-of-truth concerns. We receive already-authenticated mail; outbound retry/DSN lives *in* a channel plugin, not core; spam is the provider's job (a plugin may classify).
- **Authoritative recipient resolution.** Their directory `recipient()` decides "do I deliver to this address". Our `Address → Account` is the lighter "which of *my linked* accounts" — we are not the authoritative recipient of internet mail. We take the *shape* of their `assert_has_access` permission check for our authz seam, not its delivery semantics.

The asymmetry that is ours alone

Stalwart has one sync boundary (it is the truth → push up to clients). We have two (§State tokens & sync, "Two boundaries"): the *upward* one is identical to theirs and we copy it; the *downward* one (provider → engine reconciliation: UIDVALIDITY rolls, `historyId` gaps, re-keying) is a problem Stalwart never faces and gives us nothing for. That downward half — provenance, the channel op log, `uidValidityRolled` / `syncTokenExpired` — is our own contribution, not borrowed.

Open Questions

- Plugin language? (HTTP means any language; reference SDK in Rust + TS?)
- How do plugins discover each other's namespace schemas at runtime?
- Migration story for metadata schema bumps.
- Encrypted-at-rest: core concern or storage-plugin concern?
- Threading algorithm as plugin — but threads are referenced by core. Who owns the canonical thread id?

Resolved (see §Prior Art):

- Event bus topology — *pluggable coordinator, default in-process*; NATS/ Kafka are opt-in operator choices, not core assumptions.
- Hot reload — *yes, via atomic config/registry Arc swap*; cheap enough that restart-only is not worth the limitation.
- Webhook backpressure — *per-subscriber `lossy/lossless` drop policy* plus the DLQ (§rule 6); a slow consumer cannot stall the bus.

Stress-test Scenarios

Scenarios we will use to validate any concrete design. A design that can't express all of these cleanly is wrong.

1. **Gmail catch-all.** One domain, N synthetic addresses, all into one account, replies preserve original To.

2. **PGP-encrypted IMAP.** Bodies opaque to indexer; metadata still searchable; an encryption plugin owns key material.
3. **AI auto-responder.** Subscribes to `email.received`, writes annotation to its namespace, optionally drafts a reply via `EmailSubmission`.
4. **Split send/receive.** Receive over IMAP from provider A, send over SMTP via provider B, single logical account.
5. **Custom protocol.** Someone ships a Matrix-bridge plugin that exposes Matrix rooms as Mailboxes.
6. **Two UIs at once.** Web UI and TUI both annotate the same email; annotations don't collide because they live in separate namespaces.

Glossary

Core The fixed kernel — object model, event bus, registry, schema registry, capability seam.
Plugin Anything that extends or consumes core via the wire protocol.
Namespace A plugin's private key under `ext` in any object's metadata.
Channel A concrete protocol+credentials binding owned by a plugin.
Capability A declared, future-enforceable permission string.

Changelog

- **2026-05-21** — Initial draft. Captured: plugin-first thesis, JSON-schema HTTP+webhook wire format, UI-as-plugin, JMAP-shaped storage, routing as plugin, authz seam-only, hybrid config. Stress-test scenarios listed.
- **2026-05-21** — Added Lifecycle & State Machines section: two machines (outbound `EmailSubmission` lifecycle + per-account flags on `EmailObject`), drafts as pre-EmailSubmission artifacts, append-only `EmailSubmissionEvent` log, worked examples.
- **2026-05-21** — Added Domain Model deep dive (immutable-artifact / mutable-state split, Mailbox semantics, Thread ownership, provenance, dedup), explicit JMAP relationship section, and phased roadmap (Phases 0–6).
- **2026-05-22** — Added Real-World Quirks & Defensive Invariants section (idempotency keys, provenance, strict+fuzzy identity, op log, precise delete verbs, event retry/DLQ, clear delivery state, per-field conflict merge, plugin trust+timeouts). Extended provenance schema with provider-specific fields (UIDVALIDITY/UID, Gmail ids, cause). Split `Email` identity into `rawHash` + `fingerprint`. Replaced overloaded `delete` verb with precise set (`removeFromMailbox`/`archive`/`trash`/`expunge`/`destroyEmail`). Split `EmailSubmission.delivered` into `accepted` / `assumedDelivered` / `delivered`. Added provider-facing error codes (`rateLimited`, `authExpired`, `syncTokenExpired`, `uidValidityRolled`, `conflict`, …).
- **2026-05-22** — Added API Surface section: batched JSON-RPC envelope with backreferences, two surfaces (client + plugin) sharing one envelope, JMAP-style `Type/Verb` methods + `view`/`subscribe` additions, plugin callbacks, event bus shape, state tokens, capability-token seam, error model, atomicity & versioning rules.
- **2026-05-21** — Added End-to-End Crypto section: two-plugin split (keystore vs. crypto), canonical-storage-is-wire-form rule, encrypt-to-self default, `ext.hackermail.crypto` namespace schema, protected headers + Autocrypt, opt-in privileged search, plaintext- tainted value marker as the only real core change.
- **2026-05-21** — Renamed core types to align with JMAP: `Message` → `Email`, `MessageState` → `EmailObject`, `Submission` → `EmailSubmission`. Webhook/method names follow: `message.received` → `email.received`, etc.
- **2026-06-09** — Concretized the sync model (§State tokens & sync): change-id as a monotonic `u64` per `(account, collection)`, append-only changelog as a range-scannable subspace, `assert_state` as the named mechanism behind `stateMismatch`, container-vs-item change streams mapped onto the `Email`/`EmailObject` split, and the upward-vs-downward two-boundary framing (client-as-server, not MTA). Adapted from Stalwart's change-id machinery.
- **2026-06-09** — Split storage into four roles (`metadata`, `blob`, `index`, `cache`), each independently swappable with a named forgettable variant, so "forgettable storage" composes instead of being monolithic. Made the changelog an explicit partition of the `metadata` role. Noted BLAKE3 as an acceptable blob-key hash (address, not signature). Adapted from Stalwart's store/blob/search/in-memory split.
- **2026-06-09** — Added Deployment Profiles section: a profile is an engine-asserted, startup-checked invariant over plugin set + storage variant + token scope. Specified the `receive-only-ephemeral` profile and pinned the three semantics forgettable storage silently inverts (dedup↔TTL coupling, sync/replay authority inversion, bounded annotation lifetime). Adapted the checked-capability shape from Stalwart's `ClusterRoles`.
- **2026-06-09** — Split the event surface into two tiers (§Event bus): synchronous pre-commit *interceptors* (`transform` / `annotate` / `decide-retention`, including the keep-or-drop hook for forgettable sinks) vs. the existing asynchronous post-commit *subscribers*. Added per-subscriber `lossy`/`lossless` drop policy. Adapted the sync-hook shape from Stalwart's MTA-Hooks, with MTA verbs (reject/quarantine) stripped — a client-as-server cannot un-receive provider-accepted mail.
- **2026-06-09** — Added Prior Art: Stalwart section framing it as MTA vs. our client-as-server: what we adopt (sync machinery, storage roles, profiles, sync hooks, hot-reload, pluggable pub/sub,

BLAKE3), what we reject (re-encrypt-at-rest, the MTA pipeline, authoritative recipient resolution), and the downward-reconciliation asymmetry that is ours alone. Retired the resolved open questions (event-bus topology, hot reload, webhook backpressure).

- **2026-06-09** — Recast the `index` storage role into an **Index Provider** contract: FTS, vector/ANN, faceted, etc. as instances of one shape, each declaring the (capability-namespaced) operators it satisfies and returning ids *with scores*. Specified the vector/embedding instance (multiplicity keyed `(model, version, scope, chunkId)`, content-shared + account-scoped, `ext.hackermail.embedding` descriptor) and a single shared provider lifecycle (backfill/incremental/delete/reindex/gate/ forgettable/degraded).
- **2026-06-09** — Added the analyze-once spine: an `AnalyzedDocument` (derived, content-addressed, transient/`plaintext-tainted` artifact, sibling of the decrypted view) produced by a single post-commit analyzer that emits an `email.analyzed` event; Index Providers fan out from that rather than re-parsing/decrypting per provider.
- **2026-06-09** — Corrected the interceptor model: `decide-retention` returns a directive over a lattice (`metadata`/`blob`/`ttl`), not a keep/drop bit; composition is most-restrictive-wins under a profile ceiling; drops write body-less tombstones. Critically, separated "interceptor unavailable" from an affirmative `drop` — unavailability must *defer* (hold uncommitted, don't advance the provider cursor), never discard; `fail-by-dropping` is forbidden.
- **2026-06-09** — Added Durability & Crash Recovery section: the no-loss invariant, replayable-vs-non-replayable channels (provider-as-WAL vs. mandatory intake spool), the always-durable intake journal keyed by the §rule 1 idempotency key (the downward-path analog of the DLQ/replay), the explicit `email.receive` lifecycle (cursor advances only on finalize), and the perfect-sync-vs-zero-retention dial (`journal = durable` default; the journal stays durable even when content is forgettable).
- **2026-06-09** — Reconciled Deployment Profiles with durability: added the `journal` dial (default `durable`) as a separate axis from `storage`, and stated that "forgettable" forgets content but never the intake journal — otherwise the receive-only profile loses its crash/outage no-loss guarantee.
- **2026-06-09** — Made `Email/query`'s `filter` a provider-agnostic algebra: core structured predicates plus Index-Provider-contributed operators (`text`, `semanticNear`, …) that are capability-namespaced + declared in `using`, and that return ids *with per-provider scores* so ranking and fusion are possible.